

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ПМИГ

**Отчет по разработке программного инструмента для оценки расстояния
с помощью веб-камеры**

Студент гр. 4492

Коптяев Г. С.

Преподаватель

Клоков А.А.

Санкт-Петербург

2026

Реферат

Отчет содержит: 16 страницы, 18 рисунков, 1 приложения

Объектом исследования является конструкция корпуса полупроводникового прибора, состоящая из основания, стеклянного изолятора, медного колпачка, никелевого и алюминиевого держателей, а также латунного корпуса.

Цель работы — комплексный расчет элементов конструкции прибора на прочность под воздействием статических инерционных нагрузок и температурных полей, оценка несущей способности выводов и сопряженных цилиндрических узлов, а также разработка конструкторской документации.

В процессе работы выполнен расчет геометрических и массовых характеристик деталей прибора, определено положение центра тяжести конструкции ($x_c \approx 10,64$ мм) и вычислена эквивалентная инерционная сила ($P \approx 2,28$ Н) при коэффициенте перегрузки $k = 400$. Раскрыта статическая неопределимость системы методом сил с применением графического способа Верещагина и аналитического интегрирования по формуле Мора; получено полное схождение результатов по определению избыточной реакции ($X_1 \approx 0,698$ Н). Построены эпюры изгибающих моментов и поперечных сил, на основании которых рассчитаны максимальные эквивалентные напряжения в опасном сечении вывода из никеля ($\sigma_{max} = 93,39$ МПа).

Методом теории упругости для составных цилиндрических тел решена задача определения радиальных и окружных температурных напряжений в сопряжении «стекло – медь – латунь» при температурном перепаде $\Delta t = 10$ °С.

На основе проведенных расчетов проведена оценка прочности по третьей теории прочности. Установлено, что для всех элементов конструкции

(никелевый вывод, медный выводной штуцер, стеклянный изолятор, латунный корпус) условия прочности надежно выполняются, а фактические напряжения не превышают допускаемых значений.

Результаты работы могут быть использованы при проектировании и оценке эксплуатационной надежности полупроводниковых приборов и радиоэлектронной аппаратуры, подвергающейся динамическим и термическим воздействиям.

Содержание

1. Введение.....	5
2. Теоретическая основа и геометрическая модель.....	6
3. Программная реализация.....	8
4. Руководство пользователя.....	10
5. Результаты тестирования и ограничения.....	13
6. Заключение.....	14
Приложение А.....	15

1. Введение

Целью данной работы является создание программного инструмента для оценки расстояния до объектов на горизонтальной поверхности с использованием одной веб-камеры (на примере Logitech).

Система предназначена для разметки опасных зон (в виде двух линий на заданных дистанциях) и измерения дальности до произвольной точки кадра, при условии, что объект касается земли.

Основные задачи:

- Реализовать расчётную модель, связывающую пиксельную строку изображения с расстоянием на плоскости.
- Разработать интерактивную программу на Python с графическим интерфейсом для изменения высоты, угла наклона камеры и расстояний маркерных линий.
- Предусмотреть быструю калибровку угла наклона по одному известному объекту.
- Обеспечить удобное управление параметрами через трекбары, клавиатурный ввод и фиксацию точки мышью.

2. Теоретическая основа и геометрическая модель

В основе расчетов лежит геометрическая модель камеры, установленной на высоте H над ровной горизонтальной землей. Оптическая ось наклонена вниз от горизонтали на угол θ (tilt).

Для точки на поверхности, находящейся на горизонтальном расстоянии D , луч визирования образует с горизонтом угол α :

$$\alpha = \arctan\left(\frac{H}{D}\right)$$

Этот луч отклоняется от оптической оси на угол $\beta = \alpha - \theta$. На матрице изображения это вызывает вертикальное смещение y_{rel} относительно центра кадра c_y :

$$y_{rel} = f_y \cdot \tan \beta$$

Где f_y — фокусное расстояние по вертикали в пикселях:

$$f_y = \frac{H_{img}/2}{\tan(VFOV/2)}$$

(H_{img} — высота кадра, $VFOV$ — вертикальный угол обзора)

Обратное преобразование от пиксельной строки y к реальному расстоянию D выполняется по следующим формулам:

$$\beta = \arctan\left(\frac{y - c_y}{f_y}\right)$$

$$\alpha = \beta + \theta$$

$$D = \frac{H}{\tan(\alpha)} \quad (\text{при } \alpha > 0)$$

Если $\alpha \leq 0$, луч уходит выше горизонта, и расстояние считается бесконечностью.

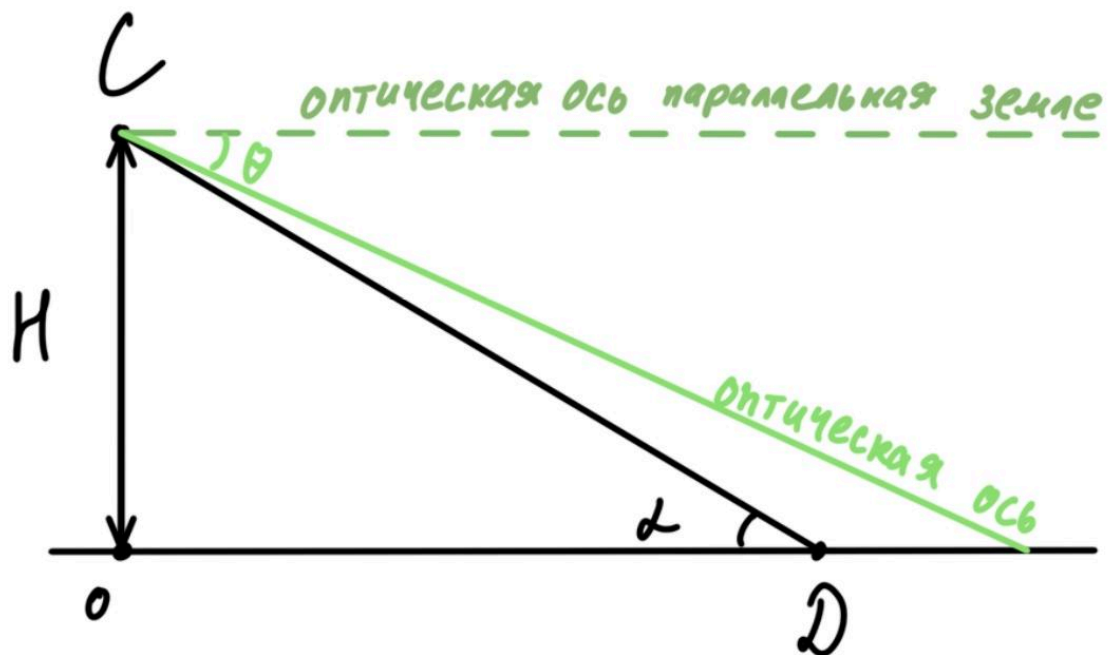


Рисунок 1: Схема геометрической модели

Калибровка угла наклона

Если известны высота H и реальное расстояние D_i до объекта, а также зафиксирована пиксельная строка касания y_i , текущий угол наклона θ вычисляется как:

$$\theta = \arctan\left(\frac{H}{D_{cal}}\right) - \arctan\left(\frac{y_{cal} - c_y}{f_y}\right)$$

Данная формула применяется для автоматической калибровки системы по одному клику мыши.

3. Программная реализация

Программа написана на языке Python 3 с использованием следующих библиотек:

OpenCV (cv2): для захвата видео (через бэкенд DirectShow для ОС Windows), создания окон, отрисовки графики и обработки событий.

NumPy: для работы с массивами изображений.

math: для вычисления тригонометрических функций.

Архитектура приложения:

Интерфейс разделен на два окна:

1. “Camera”: Вывод видеопотока с наложенными дистанционными линиями (зеленая для Dist1 и красная для Dist2), расчетным расстоянием под курсором мыши и отображением зафиксированной точки калибровки.

2. “Settings”: Содержит трекбары для настройки высоты камеры (Height, 1..300 см), угла наклона (Tilt, 0..80 град.) и дистанций маркерных линий.

По умолчанию в коде заданы параметры: $H = 178$ см, $\theta = 3030^\circ$, $VFOV = 43^\circ$. Все внутренние расчёты ведутся в метрах, но для удобства пользователя ввод и трекбары представлены в сантиметрах.

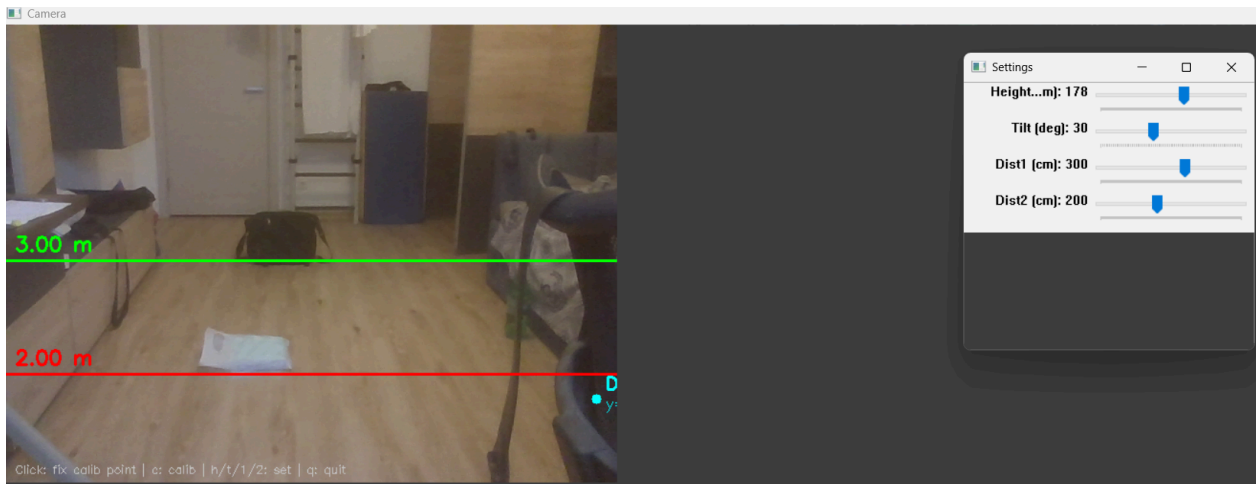


Рисунок 2: Скриншот работы программы

4. Руководство пользователя

1. Запуск: Установите библиотеки `opencv-python` и `numpy`, подключите камеру и запустите скрипт.

2. Управление: Используйте ползунки в окне “Settings” для корректировки параметров; изменения сразу отражаются на видео.

Горячие клавиши ‘h’, ‘t’, ‘1’, ‘2’ позволяют задать точные значения через консоль.

3. Измерение в реальном времени: Перемещайте курсор мыши по видеопотоку. Рядом с ним будет отображаться расчетная дистанция до точки на земле. Если указать на горизонт или выше, выведется “Dist: inf (horizon)”.

4. Калибровка:

Разместите объект на заранее известном расстоянии D_i .

Кликните левой кнопкой мыши по основанию объекта на видео (появится желтый маркер CALIB).

Нажмите клавишу “c”, затем введите в консоль высоту H и дистанцию D_i в сантиметрах.

Программа автоматически пересчитает угол θ и обновит линии.



Рисунок 3: Скриншот процесса калибровки

5. Результаты тестирования и ограничения

Программа была успешно протестирована с камерой Logitech C920 (высота установки 178 см). При калибровке по объекту на расстоянии 238 см), система вычислила угол наклона $\theta \approx 23.5$, что совпало с реальным физическим положением. Измерения в диапазоне 3-10 метров показали погрешность 3-12 см на ровной поверхности при хорошем освещении.

Ключевые ограничения метода:

- Корректная оценка дальности возможна только для объектов, непосредственно касающихся земли.
- Требуется строго ровная горизонтальная поверхность; уклоны искажают результаты.
- Недопустимы сильные вибрации камеры, изменяющие ее высоту или наклон без компенсации.

6. Заключение

Созданный инструмент на Python позволяет эффективно решать задачу монокулярной оценки расстояния с использованием доступных веб-камер. Интерактивный интерфейс и алгоритм калибровки «в один клик» делают систему удобной для быстрой настройки. Решение отлично подходит для прототипирования систем помощи водителям (ADAS), роботизированных платформ и исследовательских проектов в сфере компьютерного зрения. В перспективе инструмент может быть интегрирован с нейросетевыми детекторами объектов (например, YOLO) для полной автоматизации процесса.

Приложение А

Код для вычисления P1 и P2

```
import cv2

import numpy as np

import math

# Параметры по умолчанию
H_DEFAULT_CM = 178
TILT_DEG_DEFAULT = 30
VFOV_DEG = 43.0
FRAME_HEIGHT = 480
D1_CM_DEFAULT = 100
D2_CM_DEFAULT = 200

mouse_x, mouse_y = -1, -1
calib_y = -1
calib_x = -1

def mouse_callback(event, x, y, flags, param):
    global mouse_x, mouse_y, calib_y, calib_x
    if event == cv2.EVENT_MOUSEMOVE:
        mouse_x, mouse_y = x, y
    elif event == cv2.EVENT_LBUTTONDOWN:
        calib_x, calib_y = x, y
        print(f"Точка калибровки зафиксирована: x={x}, y={y}")

def get_fy(frame_height, vfov_deg):
    return (frame_height / 2.0) / math.tan(math.radians(vfov_deg / 2.0))

def get_y_for_distance(distance_m, H_m, tilt_deg, fy, cy):
    alpha = math.atan2(H_m, distance_m)
    beta = alpha - math.radians(tilt_deg)
    y_rel = fy * math.tan(beta)
    return cy + y_rel
```

```

def get_distance_from_y(y, H_m, tilt_deg, fy, cy):
    y_rel = y - cy
    beta = math.atan2(y_rel, fy)
    alpha = beta + math.radians(tilt_deg)
    if alpha <= 0.0:
        return float('inf')
    return H_m / math.tan(alpha)

# Вычисление угла наклона (градусы) по точке на земле
def calibrate_tilt_from_point(y, H_m, distance_m, fy, cy):
    alpha = math.atan2(H_m, distance_m)
    y_rel = y - cy
    beta = math.atan2(y_rel, fy)
    theta_rad = alpha - beta
    return math.degrees(theta_rad)

def main():
    global mouse_x, mouse_y, calib_y, calib_x

    cap = cv2.VideoCapture(1, cv2.CAP_DSHOW) # индекс камеры
    if not cap.isOpened():
        cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
        if not cap.isOpened():
            print("Ошибка: не удалось открыть камеру")
            return

    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_HEIGHT)
    real_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    real_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    print(f"Разрешение камеры: {real_width}x{real_height}")

    cy = real_height / 2.0
    fy = get_fy(real_height, VFOV_DEG)

    cv2.namedWindow("Camera")
    cv2.setMouseCallback("Camera", mouse_callback)

    cv2.namedWindow("Settings")

```

```

cv2.createTrackbar("Height (cm)", "Settings", H_DEFAULT_CM, 300, lambda x: None)
cv2.createTrackbar("Tilt (deg)", "Settings", TILT_DEG_DEFAULT, 80, lambda x: None)
cv2.createTrackbar("Dist1 (cm)", "Settings", D1_CM_DEFAULT, 500, lambda x: None)
cv2.createTrackbar("Dist2 (cm)", "Settings", D2_CM_DEFAULT, 500, lambda x: None)

print("Управление:")
print("  Левая кнопка мыши - зафиксировать точку для калибровки")
print("  с - выполнить калибровку наклона по зафиксированной точке")
print("  h / t / 1 / 2 - ввести параметры через консоль")
print("  q / ESC - выход")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    H_cm = cv2.getTrackbarPos("Height (cm)", "Settings")
    H_m = H_cm / 100.0
    tilt_deg = cv2.getTrackbarPos("Tilt (deg)", "Settings")
    dist1_cm = cv2.getTrackbarPos("Dist1 (cm)", "Settings")
    dist2_cm = cv2.getTrackbarPos("Dist2 (cm)", "Settings")
    dist1_m = dist1_cm / 100.0
    dist2_m = dist2_cm / 100.0

    y1 = get_y_for_distance(dist1_m, H_m, tilt_deg, fy, cy)
    y2 = get_y_for_distance(dist2_m, H_m, tilt_deg, fy, cy)

    h, w = frame.shape[:2]

    # Линии расстояний
    if 0 <= y1 < h:
        y1_int = int(round(y1))
        cv2.line(frame, (0, y1_int), (w, y1_int), (0, 255, 0), 2)
        cv2.putText(frame, f"{dist1_m:.2f} m", (10, y1_int - 10),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    else:
        cv2.putText(frame, f"Dist1 ({dist1_m:.2f}m) out of frame", (10, 30),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

```

```

if 0 <= y2 < h:
    y2_int = int(round(y2))
    cv2.line(frame, (0, y2_int), (w, y2_int), (0, 0, 255), 2)
    cv2.putText(frame, f"{dist2_m:.2f} m", (10, y2_int - 10),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
else:
    cv2.putText(frame, f"Dist2 ({dist2_m:.2f}m) out of frame", (10, 60),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

# Отображение зафиксированной точки калибровки
if calib_y >= 0:
    cv2.circle(frame, (calib_x, calib_y), 8, (0, 255, 255), 2)
    cv2.line(frame, (0, calib_y), (w, calib_y), (0, 255, 255), 1)
    cv2.putText(frame, f"CALIB y={calib_y}", (calib_x + 12, calib_y - 12),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 2)

# Расстояние под курсором мыши
if 0 <= mouse_x < w and 0 <= mouse_y < h:
    dist_mouse = get_distance_from_y(mouse_y, H_m, tilt_deg, fy, cy)
    cv2.circle(frame, (mouse_x, mouse_y), 5, (255, 255, 0), -1)
    if math.isfinite(dist_mouse):
        text = f"Dist: {dist_mouse:.2f} m"
    else:
        text = "Dist: inf (horizon)"
    cv2.putText(frame, text, (mouse_x + 10, mouse_y - 10),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 0), 2)
    cv2.putText(frame, f"y={mouse_y}", (mouse_x + 10, mouse_y + 10),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)

# Инструкция
cv2.putText(frame, "Click: fix calib point | c: calib | h/t/1/2: set | q:
quit",
            (10, h - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (200, 200, 200), 1)

cv2.imshow("Camera", frame)

key = cv2.waitKey(1) & 0xFF
if key == 27 or key == ord('q'):

```

```

        break

# Обработка клавиатуры
try:
    if key == ord('h'):
        val = int(input("Введите высоту камеры (см): "))
        val = max(1, min(300, val))
        cv2.setTrackbarPos("Height (cm)", "Settings", val)
    elif key == ord('t'):
        val = int(input("Введите угол наклона (градусы): "))
        val = max(0, min(80, val))
        cv2.setTrackbarPos("Tilt (deg)", "Settings", val)
    elif key == ord('l'):
        val = int(input("Введите дистанцию 1 (см): "))
        val = max(1, min(500, val))
        cv2.setTrackbarPos("Dist1 (cm)", "Settings", val)
    elif key == ord('2'):
        val = int(input("Введите дистанцию 2 (см): "))
        val = max(1, min(500, val))
        cv2.setTrackbarPos("Dist2 (cm)", "Settings", val)
    elif key == ord('c'):
        # Используем зафиксированную точку, если есть, иначе текущую позицию
        y_for_calib = calib_y if calib_y >= 0 else mouse_y
        if 0 <= y_for_calib < h:
            print(f"\nКалибровка по точке y = {y_for_calib}")
            H_calib_cm = float(input("Высота камеры (см): "))
            D_calib_cm = float(input("Расстояние до объекта (см): "))
            H_calib_m = H_calib_cm / 100.0
            D_calib_m = D_calib_cm / 100.0
            new_tilt = calibrate_tilt_from_point(y_for_calib, H_calib_m,
D_calib_m, fy, cy)
            print(f"Вычисленный угол наклона: {new_tilt:.1f}°")
            cv2.setTrackbarPos("Tilt (deg)", "Settings", int(round(new_tilt)))
            cv2.setTrackbarPos("Height (cm)", "Settings", int(H_calib_cm))
        else:
            print("Точка калибровки не задана или вне кадра. Кликните левой
кнопкой мыши на объекте.")
    except ValueError:

```



```
        print("Ошибка: введите число.")

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```